

Documentación Técnica de Ingeniería

Despliegue de Infraestructura Cloud e Integración de Proxy Inverso para el Orquestador Core (n8n Hub)

Proyecto:	FamillA / Senda MVP	Entorno:	Producción Cloud (DigitalOcean)
Fecha:	Mayo 2026	Arquitectura:	Docker Monorepo / Reverse Proxy TLS

1. Introducción y Arquitectura de Referencia

Este documento detalla el procedimiento de ingeniería implementado para el despliegue del nodo orquestador del proyecto **FamillA**. El objetivo es consolidar un entorno de ejecución persistente, seguro y de alta disponibilidad, minimizando los costes operativos mediante el aprovechamiento de recursos de desarrollo cloud.

La topología del sistema se basa en un servidor virtual privado (VPS) configurado bajo arquitectura Linux, donde el motor de orquestación (n8n) opera aislado dentro de un contenedor Docker, expuesto al exterior exclusivamente a través de una capa intermedia de enrutamiento seguro gestionada por un proxy inverso automático.

2. Fase 1: Criptografía y Acceso Seguro E2E (WSL2 a VPS)

Para erradicar vectores de ataque por fuerza bruta o interceptación de credenciales en tránsito, se ha eliminado por completo la autenticación por contraseña tradicional del servidor, sustituyéndola por un protocolo criptográfico asimétrico de clave pública.

2.1 Generación del Par de Claves en el Entorno Local

Operando desde el entorno local bajo **WSL2 (Windows Subsystem for Linux)** con la shell Zsh, se ejecutó la generación de llaves mediante el algoritmo de curva elíptica `Ed25519`. Este algoritmo ofrece un rendimiento superior y una resistencia criptográfica significativamente mayor en comparación con arquitecturas RSA obsoletas.

```
$ ssh-keygen -t ed25519 -C "vnm-hub-access"
```

El comando genera dos archivos en el directorio protegido del usuario local:

- `~/.ssh/id_ed25519` : Clave privada (permanece estrictamente confidencial en la máquina local).
- `~/.ssh/id_ed25519.pub` : Clave pública o vector de verificación.

2.2 Exportación mediante Interoperabilidad de Capas

Para transferir el vector de clave pública al portapapeles del sistema operativo anfitrión (Windows) sin corromper caracteres ni introducir saltos de línea inválidos, se explotó la tubería de interoperabilidad de binarios de WSL2:

```
$ cat ~/.ssh/id_ed25519.pub | clip.exe
```

Este flujo inyecta el contenido exacto de la clave en el panel de aprovisionamiento, permitiendo que el servidor nazca con la clave pública pre-sembrada en el archivo interno del sistema `/root/.ssh/authorized_keys`.

3. Fase 2: Dimensionamiento y Justificación de Infraestructura

El dimensionamiento del hardware se ha calculado con base en la naturaleza del consumo de recursos que realiza n8n al procesar flujos complejos con grandes cargas de datos y payloads binarios.

Componente	Configuración Seleccionada	Justificación Técnica de Ingeniería
Perfil de CPU	Premium Intel vCPU	Mejor rendimiento mononúcleo para operaciones de cifrado TLS rápidas y menor latencia en bucles Node.js.
Memoria RAM	2 GB RAM	Mínimo vital. Los procesos de Node.js en n8n cargan cargas útiles JSON y flujos binarios de audio en memoria. 1 GB causaría colapsos por <i>Out of Memory (OOM)</i> .
Almacenamiento	NVMe SSD (Capa Premium)	Crítico. n8n realiza operaciones intensivas de I/O en disco al persistir el historial detallado de cada nodo y subflujo ejecutado. Los SSD estándar colapsarían la base de datos interna.

Nota de Viabilidad Presupuestaria

Esta configuración representa un coste operativo mensual parametrizado en ~14-16 USD, garantizando una autonomía ininterrumpida de más de 12 meses consumiendo de forma exacta los 200 USD de crédito otorgados por el GitHub Student Developer Pack.

4. Fase 3: Mitigación de Bloqueos de Red y Estrategia de Proxy Inverso

Por defecto, n8n escucha de forma nativa en el puerto `5678`. No obstante, las políticas de seguridad perimetral de las redes académicas y corporativas (como Eduroam) aplican un filtrado estricto de paquetes de salida, capando cualquier conexión que no se dirija a los puertos web estándar (`80` para HTTP y `443` para HTTPS). Esto hacía imposible la conexión directa al puerto por defecto desde el entorno de desarrollo habitual.

4.1 Resolución de Certificados SSL sin Dominio de Nivel Superior

Las entidades certificadoras no emiten certificados de seguridad criptográfica TLS/SSL para direcciones IP públicas crudas, requiriendo obligatoriamente un Nombre de Dominio Totalmente Calificado (FQDN). Para resolver esta restricción de forma nativa desde el propio Droplet, se recurrió a un mapeo dinámico inverso utilizando el servicio **sslip.io**.

Al configurar la máquina bajo el subdominio estructurado como `[IP_PUBLICA].sslip.io`, los servidores DNS mundiales resuelven dicha cadena devolviendo de manera exacta la IP del Droplet. Esto constituye un nombre de dominio válido ante la API de Let's Encrypt, permitiendo la emisión automática de certificados SSL sin costes añadidos.

4.2 Implementación de Caddy como Proxy Inverso

Para gestionar el tráfico de entrada y encapsular el cifrado, se desplegó **Caddy** en la frontera de la red. Caddy actúa interceptando las peticiones seguras en el puerto `443`, validando el cifrado del lado del cliente, y redirigiendo la petición de forma limpia y transparente al puerto interno `5678` del contenedor de n8n a través de la red aislada de Docker.

5. Fase 4: Definición de la Infraestructura como Código (Docker Compose)

Toda la capa de servicios e infraestructura se declaró en un manifiesto único de orquestación en la ruta `~/familia-hub/docker-compose.yml`. A continuación se expone el código de configuración de producción final implementado:

```
volumes:
  n8n_data:
  caddy_data:
  caddy_config:

services:
  caddy:
    image: caddy:latest
    restart: always
    ports:
      - "80:80"
      - "443:443"
    volumes:
```

```

- caddy_data:/data
- caddy_config:/config
command: caddy reverse-proxy --from https://209.38.213.186.sslip.io --to n8n:5678

n8n:
  image: docker.n8n.io/n8nio/n8n
  restart: always
  environment:
    - NODE_ENV=production
    - WEBHOOK_URL=https://209.38.213.186.sslip.io/
    - GENERIC_TIMEZONE=Europe/Madrid
    - EXECUTIONS_DATA_PRUNE=true
    - EXECUTIONS_DATA_MAX_AGE=168
  volumes:
    - n8n_data:/home/node/.n8n

```

5.1 Análisis de Parámetros Críticos del Contenedor

- **Aislamiento de Red:** Se eliminó la directiva `ports` expuesta en el servicio de n8n. Esto blindo el backend, impidiendo que nadie pueda saltarse el proxy seguro o atacar el puerto nativo 5678 directamente.
- `EXECUTIONS_DATA_PRUNE=true` y `EXECUTIONS_DATA_MAX_AGE=168` : Directivas de optimización indispensables para entornos con memoria limitada a 2 GB. Fuerza a la base de datos interna a purgar el historial de ejecuciones de forma automática cada 7 días (168 horas), previniendo la degradación del almacenamiento y colapsos del sistema por saturación de logs.

6. Fase 5: Configuración del Cortafuegos del Sistema Operativo (UFW)

Para garantizar que el tráfico no autorizado sea descartado a nivel de kernel, se configuró el módulo `Uncomplicated Firewall (UFW)` de Ubuntu, autorizando explícitamente solo los vectores de entrada legítimos:

```

# Permitir el tráfico web estándar y el protocolo seguro para Caddy
# ufw allow 80/tcp
# ufw allow 443/tcp

# Aplicar las políticas de filtrado y reiniciar el daemon del firewall
# ufw reload

```

Esta configuración descarta cualquier intento de escaneo o conexión dirigida a puertos internos del ecosistema Docker, delegando toda la responsabilidad de validación perimetral en la capa de Caddy. Con esto concluye el despliegue funcional de la infraestructura base del entorno.